

```

for (i=1;i<(n>>2);i++) {
    i2=1+(i1=i+i);
    i4=1+(i3=n-i1);
    h1r=c1*(data[i1]+data[i3]);
    h1i=c1*(data[i2]-data[i4]);
    h2r= -c2*(data[i2]+data[i4]);
    h2i=c2*(data[i1]-data[i3]);
    data[i1]=h1r+wr*h2r-wi*h2i;
    data[i2]=h1i+wr*h2i+wi*h2r;
    data[i3]=h1r-wr*h2r+wi*h2i;
    data[i4]= -h1i+wr*h2i+wi*h2r;
    wr=(wtemp=wr)*wpr-wi*wpi+wr;
    wi=wi*wpr+wtemp*wpi+wi;
}
if (isign == 1) {
    data[0] = (h1r=data[0])+data[1];
    data[1] = h1r-data[1];
} else {
    data[0]=c1*((h1r=data[0])+data[1]);
    data[1]=c1*(h1r-data[1]);
    four1(data,-1);
}
}

```

Case i=0 done separately below.

The two separate transforms are separated out of data.

Here they are recombined to form the true transform of the original real data.

The recurrence.

Squeeze the first and last data together to get them all within the original array.

This is the inverse transform for the case isign=-1.

You can't use `WrapVecDoub` (§12.2) to access the output of `realft` as complex values; it assumes a wraparound order that is not valid when we are storing only the positive part of the spectrum. An even simpler trick, however, is to define an inline function

```
inline Complex* Cmplx(VecDoub &d) {return (Complex *)&d[0];}
```

and then write things like

```

Cmplx(data)[k] = Complex(... , ...);
cvalue = Cmplx(data)[k];

```

when you want to set or get the k^{th} complex value in `data`, viewed as a complex array. (You are still responsible for separating the two real values stored in the first complex component, however.)

CITED REFERENCES AND FURTHER READING:

- Brigham, E.O. 1974, *The Fast Fourier Transform* (Englewood Cliffs, NJ: Prentice-Hall), §10-10.
- Sorensen, H.V., Jones, D.L., Heideman, M.T., and Burris, C.S. 1987, "Real-Valued Fast Fourier Transform Algorithms," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. ASSP-35, pp. 849–863.
- Hockney, R.W. 1971, in *Methods in Computational Physics*, vol. 9 (New York: Academic Press).
- Russ, J.C. 2002, *The Image Processing Handbook*, 4th ed. (Boca Raton, FL: CRC Press)
- Clarke, R.J. 1985, *Transform Coding of Images*, (Reading, MA: Addison-Wesley).
- Gonzalez, R.C., and Woods, R.E. 1992, *Digital Image Processing*, 2nd ed. (Reading, MA: Addison-Wesley).

12.4 Fast Sine and Cosine Transforms

Among their other uses, the Fourier transforms of functions can be used to solve differential equations (see §20.4). The most common boundary conditions for the solutions are (i)

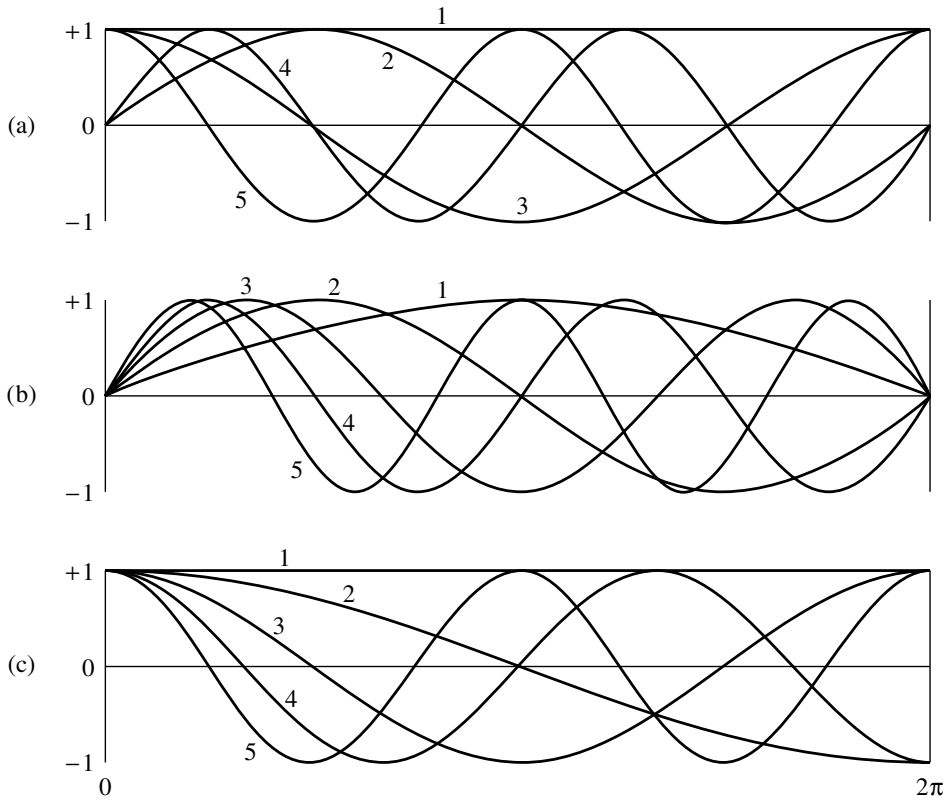


Figure 12.4.1. Basis functions used by the Fourier transform (a), sine transform (b), and cosine transform (c) are plotted. The first five basis functions are shown in each case. (For the Fourier transform, the real and imaginary parts of the basis functions are both shown.) While some basis functions occur in more than one transform, the basis sets are distinct. For example, the sine transform functions labeled (1), (3), (5) are not present in the Fourier basis. Any of the three sets can expand any function in the interval shown; however, the sine or cosine transform best expands functions matching the boundary conditions of the respective basis functions, namely zero function values for sine, zero derivatives for cosine.

they have the value zero at the boundaries, or (ii) their derivatives are zero at the boundaries. In the first instance, the natural transform to use is the *sine transform*, while in the second, one of several variations of the *cosine transform* is a natural choice.

12.4.1 Sine Transform

The sine transform is given by

$$F_k = \sum_{j=1}^{N-1} f_j \sin(\pi j k / N) \quad (12.4.1)$$

where f_j , $j = 0, \dots, N-1$ is the data array, and $f_0 \equiv 0$.

At first blush this appears to be simply the imaginary part of the discrete Fourier transform. However, the argument of the sine differs by a factor of two from the value that would make this so. The sine transform uses *sines only* as a complete set of functions in the interval from 0 to 2π , and, as we shall see, the cosine transform uses *cosines only*. By contrast, the normal FFT uses both sines and cosines, but only half as many of each. (See Figure 12.4.1.)

The expression (12.4.1) can be “force-fit” into a form that allows its calculation via the FFT. The idea is to extend the given function rightward past its last tabulated value. We extend

the data to twice their length in such a way as to make them an *odd* function about $j = N$, with $f_N = 0$,

$$f_{2N-j} \equiv -f_j \quad j = 0, \dots, N-1 \quad (12.4.2)$$

Consider the FFT of this extended function:

$$F_k = \sum_{j=0}^{2N-1} f_j e^{2\pi i j k / (2N)} \quad (12.4.3)$$

The half of this sum from $j = N$ to $j = 2N-1$ can be rewritten with the substitution $j' = 2N - j$,

$$\begin{aligned} \sum_{j=N}^{2N-1} f_j e^{2\pi i j k / (2N)} &= \sum_{j'=1}^N f_{2N-j'} e^{2\pi i (2N-j')k / (2N)} \\ &= - \sum_{j'=0}^{N-1} f_{j'} e^{-2\pi i j' k / (2N)} \end{aligned} \quad (12.4.4)$$

so that

$$\begin{aligned} F_k &= \sum_{j=0}^{N-1} f_j \left[e^{2\pi i j k / (2N)} - e^{-2\pi i j k / (2N)} \right] \\ &= 2i \sum_{j=0}^{N-1} f_j \sin(\pi j k / N) \end{aligned} \quad (12.4.5)$$

Thus, up to a factor $2i$ we get the sine transform from the FFT of the extended function.

This method introduces a factor of two inefficiency into the computation by extending the data. This inefficiency shows up in the FFT output, which has zeros for the real part of every element of the transform. For a one-dimensional problem, the factor of two may be bearable, especially in view of the simplicity of the method. When we work with partial differential equations in two or three dimensions, though, the factor becomes four or eight, so efforts to eliminate the inefficiency are well rewarded.

From the original real data array f_j we will construct an auxiliary array y_j and apply to it the routine `realft`. The output will then be used to construct the desired transform. For the sine transform of data f_j , $j = 1, \dots, N-1$, the auxiliary array is

$$\begin{aligned} y_0 &= 0 \\ y_j &= \sin(j\pi/N)(f_j + f_{N-j}) + \frac{1}{2}(f_j - f_{N-j}) \quad j = 1, \dots, N-1 \end{aligned} \quad (12.4.6)$$

This array is of the same dimension as the original. Notice that the first term is symmetric about $j = N/2$ and the second is antisymmetric. Consequently, when `realft` is applied to y_j , the result has real parts R_k and imaginary parts I_k given by

$$\begin{aligned} R_k &= \sum_{j=0}^{N-1} y_j \cos(2\pi j k / N) \\ &= \sum_{j=1}^{N-1} (f_j + f_{N-j}) \sin(j\pi/N) \cos(2\pi j k / N) \\ &= \sum_{j=0}^{N-1} 2f_j \sin(j\pi/N) \cos(2\pi j k / N) \end{aligned}$$

$$\begin{aligned}
&= \sum_{j=0}^{N-1} f_j \left[\sin \frac{(2k+1)j\pi}{N} - \sin \frac{(2k-1)j\pi}{N} \right] \\
&= F_{2k+1} - F_{2k-1}
\end{aligned} \tag{12.4.7}$$

$$\begin{aligned}
I_k &= \sum_{j=0}^{N-1} y_j \sin(2\pi jk/N) \\
&= \sum_{j=1}^{N-1} (f_j - f_{N-j}) \frac{1}{2} \sin(2\pi jk/N) \\
&= \sum_{j=0}^{N-1} f_j \sin(2\pi jk/N) \\
&= F_{2k}
\end{aligned} \tag{12.4.8}$$

Therefore, F_k can be determined as follows:

$$F_{2k} = I_k \quad F_{2k+1} = F_{2k-1} + R_k \quad k = 0, \dots, (N/2 - 1) \tag{12.4.9}$$

The even terms of F_k are thus determined very directly. The odd terms require a recursion, the starting point of which follows from setting $k = 0$ in equation (12.4.9) and using $F_1 = -F_{-1}$:

$$F_1 = \frac{1}{2} R_0 \tag{12.4.10}$$

The implementing program is

```

void sinft(VecDoub_IO &y) {
    Calculates the sine transform of a set of n real-valued data points stored in array y[0..n-1].
    The number n must be a power of 2. On exit, y is replaced by its transform. This program,
    without changes, also calculates the inverse sine transform, but in this case the output array
    should be multiplied by 2/n.
    Int j,n=y.size();
    Doub sum,y1,y2,theta,wi=0.0,wr=1.0,wpi,wpr,wtemp;
    theta=3.141592653589793238/Doub(n);           Initialize the recurrence.
    wtemp=sin(0.5*theta);
    wpr= -2.0*wtemp*wtemp;
    wpi=sin(theta);
    y[0]=0.0;
    for (j=1;j<(n>>1)+1;j++) {
        wr=(wtemp*wr)*wpr-wi*wpi+wr;      Calculate the sine for the auxiliary array.
        wi=wi*wpr+wtemp*wpi+wi;           The cosine is needed to continue the recurrence.
        y1=wi*(y[j]+y[n-j]);               Construct the auxiliary array.
        y2=0.5*(y[j]-y[n-j]);
        y[j]=y1+y2;
        y[n-j]=y1-y2;
    }
    reallft(y,1);
    y[0]*=0.5;
    sum=y[1]=0.0;
    for (j=0;j<n-1;j+=2) {
        sum += y[j];
        y[j]=y[j+1];
        y[j+1]=sum;
    }
}

```

fourier.h

Calculate the sine for the auxiliary array.
The cosine is needed to continue the recurrence.
Construct the auxiliary array.

Terms j and $N - j$ are related.

Transform the auxiliary array.
Initialize the sum used for odd terms below.

Even terms determined directly.
Odd terms determined by this running sum.

The sine transform, curiously, is its own inverse. If you apply it twice, you get the original data, but multiplied by a factor of $N/2$.

12.4.2 Cosine Transform

The other common boundary condition for differential equations is that the derivative of the function is zero at the boundary. In this case, the natural transform is the *cosine* transform. There are several possible ways of defining the transform. Each can be thought of as resulting from a different way of extending a given array to create an even array of double the length, and/or from whether the extended array contains $2N-1$, $2N$, or some other number of points. In practice, only two of the numerous possibilities are useful, so we will restrict ourselves to just these two.

The first form of the cosine transform uses $N+1$ data points:

$$F_k = \frac{1}{2}[f_0 + (-1)^k f_N] + \sum_{j=1}^{N-1} f_j \cos(\pi j k / N) \quad (12.4.11)$$

It results from extending the given array to an even array about $j = N$, with

$$f_{2N-j} = f_j, \quad j = 0, \dots, N-1 \quad (12.4.12)$$

If you substitute this extended array into equation (12.4.3) and follow steps analogous to those leading up to equation (12.4.5), you will find that the Fourier transform is just twice the cosine transform (12.4.11). Another way of thinking about the formula (12.4.11) is to notice that it is the Chebyshev-Gauss-Lobatto quadrature formula (see §4.6), often used in Clenshaw-Curtis adaptive quadrature (see §5.9, equation 5.9.4).

Once again the transform can be computed without the factor of two inefficiency. In this case the auxiliary function is

$$y_j = \frac{1}{2}(f_j + f_{N-j}) - \sin(j\pi/N)(f_j - f_{N-j}) \quad j = 0, \dots, N-1 \quad (12.4.13)$$

Instead of equation (12.4.9), `realft` now gives

$$F_{2k} = R_k \quad F_{2k+1} = F_{2k-1} + I_k \quad k = 0, \dots, (N/2-1) \quad (12.4.14)$$

The starting value for the recursion for odd k in this case is

$$F_1 = \frac{1}{2}(f_0 - f_N) + \sum_{j=1}^{N-1} f_j \cos(j\pi/N) \quad (12.4.15)$$

This sum does not appear naturally among the R_k and I_k , and so we accumulate it during the generation of the array y_j .

Once again this transform is its own inverse, and so the following routine works for both directions of the transformation. Note that although this form of the cosine transform has $N+1$ input and output values, it passes an array only of length N to `realft`.

`fourier.h`

```
void cosft1(VecDoub_IO &y) {
```

Calculates the cosine transform of a set $y[0..n]$ of real-valued data points. The transformed data replace the original data in array y . n must be a power of 2. This program, without changes, also calculates the inverse cosine transform, but in this case the output array should be multiplied by $2/n$.

```
    const Doub PI=3.141592653589793238;
```

```
    Int j,n=y.size()-1;
```

```
    Doub sum,y1,y2,theta,wi=0.0,wpi,wpr,wr=1.0,wtemp;
```

```
    VecDoub yy(n);
```

```
    theta=PI/n;
```

```
    wtemp=sin(0.5*theta);
```

```
    wpr = -2.0*wtemp*wtemp;
```

```
    wpi=sin(theta);
```

```
    sum=0.5*(y[0]-y[n]);
```

```
    yy[0]=0.5*(y[0]+y[n]);
```

```
    for (j=1;j<n/2;j++) {
```

Need array of length n , not $n+1$, for `realft`.

Initialize the recurrence.

```

    wr=(wtemp=wr)*wpr-wi*wpi+wr;
    wi=wi*wpr+wtemp*wpi+wi;
    y1=0.5*(y[j]+y[n-j]);
    y2=(y[j]-y[n-j]);
    yy[j]=y1-wi*y2;
    yy[n-j]=y1+wi*y2;
    sum += wr*y2;
}
yy[n/2]=y[n/2];
realft(yy,1);
for (j=0;j<n;j++) y[j]=yy[j];
y[n]=y[1];
y[1]=sum;
for (j=3;j<n;j+=2) {
    sum += y[j];
    y[j]=sum;
}
}

```

Carry out the recurrence.

Calculate the auxiliary function.

The values for j and $N - j$ are related.

Carry along this sum for later use in unfolding the transform.

$y[n/2]$ unchanged.

Calculate the transform of the auxiliary function.

sum is the value of F_1 in equation (12.4.15).

Equation (12.4.14).

The second important form of the cosine transform is defined by

$$F_k = \sum_{j=0}^{N-1} f_j \cos \frac{\pi k(j + \frac{1}{2})}{N} \quad (12.4.16)$$

with inverse

$$f_j = \frac{2}{N} \sum_{k=0}^{N-1} F_k \cos \frac{\pi k(j + \frac{1}{2})}{N} \quad (12.4.17)$$

Here the prime on the summation symbol means that the term for $k = 0$ has a coefficient of $\frac{1}{2}$ in front. This form arises by extending the given data, defined for $j = 0, \dots, N-1$, to $j = N, \dots, 2N-1$ in such a way that they are even about the point $N - \frac{1}{2}$ and periodic. (They are therefore also even about $j = -\frac{1}{2}$.) The form (12.4.17) is related to Gauss-Chebyshev quadrature (see equation 4.6.19), to Chebyshev approximation (§5.8, equation 5.8.7), and Clenshaw-Curtis quadrature (§5.9).

This form of the cosine transform is useful when solving differential equations on “staggered” grids, where the variables are centered midway between mesh points. It is also the standard form in the field of data compression and image processing.

The auxiliary function used in this case is similar to equation (12.4.13):

$$y_j = \frac{1}{2}(f_j + f_{N-j-1}) + \sin \frac{\pi(j + \frac{1}{2})}{N}(f_j - f_{N-j-1}) \quad j = 0, \dots, N-1 \quad (12.4.18)$$

Carrying out the steps similar to those used to get from (12.4.6) to (12.4.9), we find

$$F_{2k} = \cos \frac{\pi k}{N} R_k - \sin \frac{\pi k}{N} I_k \quad (12.4.19)$$

$$F_{2k-1} = \sin \frac{\pi k}{N} R_k + \cos \frac{\pi k}{N} I_k + F_{2k+1} \quad (12.4.20)$$

Note that equation (12.4.20) gives

$$F_{N-1} = \frac{1}{2} R_{N/2} \quad (12.4.21)$$

Thus the even components are found directly from (12.4.19), while the odd components are found by recursing (12.4.20) down from $k = N/2 - 1$, using (12.4.21) to start.

Since the transform is not self-inverting, we have to reverse the above steps to find the inverse. Here is the routine:

fourier.h

```

void cosft2(VecDoub_IO &y, const Int isign) {
    Calculates the "staggered" cosine transform of a set y[0..n-1] of real-valued data points. The
    transformed data replace the original data in array y. n must be a power of 2. Set isign to
    +1 for a transform, and to -1 for an inverse transform. For an inverse transform, the output
    array should be multiplied by 2/n.
    const Doub PI=3.141592653589793238;
    Int i,n=y.size();
    Doub sum,sum1,y1,y2,ytemp,theta,wi=0.0,wi1,wpi,wpr,wr=1.0,wr1,wtemp;
    theta=0.5*PI/n;
    wr1=cos(theta);
    wi1=sin(theta);
    wpr = -2.0*wi1*wi1;
    wpi=sin(2.0*theta);
    if (isign == 1) {
        Forward transform.
        for (i=0;i<n/2;i++) {
            Calculate the auxiliary function.
            y1=0.5*(y[i]+y[n-1-i]);
            y2=wi1*(y[i]-y[n-1-i]);
            y[i]=y1+y2;
            y[n-1-i]=y1-y2;
            wr1=(wtemp=wr1)*wpr-wi1*wpi+wr1;
            wi1=wi1*wpr+wtemp*wpi+wi1;
            Carry out the recurrence.
        }
        realft(y,1);
        Transform the auxiliary function.
        for (i=2;i<n;i+=2) {
            Even terms.
            wr=(wtemp=wr)*wpr-wi*wpi+wr;
            wi=wi*wpr+wtemp*wpi+wi;
            y1=y[i]*wr-y[i+1]*wi;
            y2=y[i+1]*wr+y[i]*wi;
            y[i]=y1;
            y[i+1]=y2;
        }
        sum=0.5*y[1];
        Initialize recurrence for odd terms
        for (i=n-1;i>0;i-=2) {
            with  $\frac{1}{2}R_{N/2}$ .
            sum1=sum;
            sum += y[i];
            Carry out recurrence for odd terms.
            y[i]=sum1;
        }
    } else if (isign == -1) {
        Inverse transform.
        ytemp=y[n-1];
        Form difference of odd terms.
        for (i=n-1;i>2;i-=2)
            y[i]=y[i-2]-y[i];
        y[1]=2.0*ytemp;
        Calculate  $R_k$  and  $I_k$ .
        for (i=2;i<n;i+=2) {
            wr=(wtemp=wr)*wpr-wi*wpi+wr;
            wi=wi*wpr+wtemp*wpi+wi;
            y1=y[i]*wr+y[i+1]*wi;
            y2=y[i+1]*wr-y[i]*wi;
            y[i]=y1;
            y[i+1]=y2;
        }
        realft(y,-1);
        Invert auxiliary array.
        for (i=0;i<n/2;i++) {
            y1=y[i]+y[n-1-i];
            y2=(0.5/wi1)*(y[i]-y[n-1-i]);
            y[i]=0.5*(y1+y2);
            y[n-1-i]=0.5*(y1-y2);
            wr1=(wtemp=wr1)*wpr-wi1*wpi+wr1;
            wi1=wi1*wpr+wtemp*wpi+wi1;
        }
    }
}

```

An alternative way of implementing this algorithm is to form an auxiliary function by